

AAMP - MonitorAV Feature Summary

Purpose

MonitorAV is a background diagnostic feature that periodically samples GStreamer audio/video pipeline positions to detect and classify playback health issues in real time. When enabled it fires AAMP_EVENT_MONITORAV_STATUS events carrying a status label, AV positions, time-in-state, and dropped-frame count.

Configuration (AampConfig.h / AampConfig.cpp)

Config Key	Default	Description
eAAMPConfig_MonitorAV (monitorAV)	false	Master enable/disable flag
eAAMPConfig_MonitorAVReportingInterval (monitorAVReportingInterval)	1000 ms	Cadence of AAMP_EVENT_MONITORAV_STATUS events
eAAMPConfig_MonitorAVSyncThresholdPositive (monitorAVSyncThresholdPositive)	100 ms	Max video-ahead-of-audio before "avsync" state
eAAMPConfig_MonitorAVSyncThresholdNegative (monitorAVSyncThresholdNegative)	100 ms	Max audio-ahead-of-video before "avsync" state
eAAMPConfig_MonitorAVJumpThreshold (monitorAVJumpThreshold)	100 ms	Min unexpected video advance per interval before "jump" state

Data Structure - struct MonitorAVState

Defined in: middleware/InterfacePlayerRDK.h (line 45)

Stored in: GstPrivateContext::monitorAVstate (middleware/InterfacePlayerPriv.h, line 167)

Field	Type	Description
tLastReported	int64_t	Timestamp (ms) when description last changed and was logged
tLastSampled	int64_t	Timestamp (ms) of the most recent sampling pass
description	const char *	Current status label: "ok", "video freeze", "audio drop", "stall", "avsync", "jump", "trickplay", "eos"
av_position[2]	signed long [2]	Last known video (index 0) and audio (index 1) positions in ms
happy	bool	True once both tracks have advanced; gates jump detection

Sampling - MonitorAV() (middleware/InterfacePlayerRDK.cpp, line 634)

Called inside the existing GLib progress timer callback (ProgressCallbackOnTimeout) so it runs at the same cadence as normal playback-progress events - no extra timer needed.

```
ProgressCallbackOnTimeout()    // GLib periodic timer
    if (m_gstConfigParam->monitorAV)
```

AAMP - MonitorAV Feature Summary

```
MonitorAV(pInterfacePlayerRDK);    // sample & classify
TriggerEvent(progressCb);
```

GStreamer APIs used inside MonitorAV():

GStreamer API	Purpose
<code>gst_element_get_state(pipeline, &state, &pending, /*timeout=*/0)</code>	Non-blocking state check - sampling only runs when GST_STATE_PLAYING
<code>gst_element_query_position(sinkbin, GST_FORMAT_TIME, &position)</code>	Query decoded playback position from each sink element (video sinkbin at index 0, audio sinkbin at index 1)
<code>GST_TIME_AS_MSECONDS(position)</code>	Convert GstClockTime (nanoseconds) to milliseconds

Classification logic:

Condition	Status Label
Both tracks' positions unchanged since last sample	"stall"
Only video position unchanged	"video freeze"
Only audio position unchanged	"audio drop"
Exactly 1 active track (trickplay - audio query skipped)	"trickplay"
No active tracks	"eos"
$ \text{video_pos} - \text{audio_pos} > \text{positive or negative threshold}$	"avsync"
Video advanced $> \text{wall-clock delta} + \text{JUMP_THRESHOLD_MS}$	"jump"
None of the above	"ok"

A state-transition log (MW_LOG_MIL) is emitted whenever description changes, recording previous positions, AV delta, and time spent in the old state.

Reporting Timer - MonitorAvTimerCallback (aampgstplayer.cpp, line 1292)

A second, independent GLib timer fires at mMonitorAVInterval (default 1000 ms). It reads the latest MonitorAVState (no mutex needed - both timers run on the GLib main thread) and dispatches AAMP_EVENT_MONITORAV_STATUS.

GLib / GStreamer API	Purpose
<code>g_timeout_add(mMonitorAVInterval, MonitorAvTimerCallback, this)</code>	Register periodic reporting timer (AAMP GstPlayer::StartMonitorAvTimer, aampgstplayer.cpp line 1332)
<code>g_source_remove(monitorAvTimerId)</code>	Cancel the timer on teardown (AAMP GstPlayer::StopMonitorAvTimer, aampgstplayer.cpp line 1352)
<code>GetVideoPlaybackQuality()</code>	Retrieve dropped-frame count included in each event

Timer lifecycle:

```
Playback start -> StartMonitorAvTimer()    // aampgstplayer.cpp line 847
```

AAMP - MonitorAV Feature Summary

```
g_timeout_add(interval, MonitorAvTimerCallback, this)
```

```
Teardown      -> StopMonitorAvTimer()    // aampgstplayer.cpp line 883
                g_source_remove(monitorAvTimerId)
```

Event Delivery

PrivateInstanceAAMP::SendMonitorAvEvent() (priv_aamp.cpp, line 14977)

Constructs and dispatches an AAMP_EVENT_MONITORAV_STATUS event with:

Field	Source
status (string)	monitorAVState.description ("ok", "stall", "avsync", ...)
videoPositionMS (int64_t)	monitorAVState.av_position[eMEDIATYPE_VIDEO]
audioPositionMS (int64_t)	monitorAVState.av_position[eMEDIATYPE_AUDIO]
timeInStateMS (uint64_t)	tLastSampled - tLastReported (capped to reporting interval)
droppedFrames (uint64_t)	GstPlaybackQualityStruct::dropped

Consumed by the application via the UVE onMonitorAvStatus callback. In aampcli: test/aampcli/Aampcli.cpp, line 743.

Key File Locations

File	Relevance
middleware/InterfacePlayerRDK.h	MonitorAVState struct definition (line 45)
middleware/InterfacePlayerRDK.cpp	MonitorAV() - sampling & classification (line 634) ProgressCallbackOnTimeout - calls MonitorAV (line 763) GetMonitorAVState() accessor (line 586)
middleware/InterfacePlayerPriv.h	GstPrivateContext::monitorAVstate storage (line 167)
aampgstplayer.cpp	MonitorAvTimerCallback (line 1292) StartMonitorAvTimer / StopMonitorAvTimer (lines 1332, 1352) Config params copied to InterfacePlayer (line 98)
priv_aamp.cpp	SendMonitorAvEvent() - event dispatch (line 14977)
AampConfig.h / .cpp	eAAMPConfig_MonitorAV and related config keys
AAMP-UVE-API.md	Public API documentation for monitorAV options (line 287)
test/aampcli/Aampcli.cpp	AAMP_EVENT_MONITORAV_STATUS consumer example (line 743)